Network Intrusion Detection System (NIDS) using Snort

Design, Implementation and Analysis in a Virtual Test Lab

Table of Contents

1. Abstract 2
2. Introduction 2
   2.1 Problem statement 2
   2.2 Objectives 2
   2.3 Scope 2
3. Background 2
4. Lab Architecture 3
5. Implementation 3
   5.1 Environment setup 3
   5.2 Snort installation 3
   5.3 Configuration 3
   5.4 Running Snort 4
6. Custom Detection Rules 4
7. Attack Simulation 4
8. Results and Analysis 5
   8.1 Detection summary 5
   8.2 Log analysis 6
   8.3 Anatomy of a sample alert 6
   8.4 Discussion 6
9. Limitations 7
10. Recommendations and Future Work 7
11. Conclusion 8
12. References 8

Appendix A: Project files 8

1. Abstract

This project builds a small virtual test lab to demonstrate how a Network Intrusion Detection System (NIDS) detects malicious activity. Three virtual machines are used: an attacker (Kali Linux), a victim (Ubuntu / Metasploitable), and a sensor running Snort. Common attacks such as ping sweeps, Nmap port scans (SYN, NULL, FIN, XMAS), SYN and UDP floods, SSH brute force, and HTTP directory traversal are simulated. Twelve custom Snort rules are written to detect them. Alerts are collected, parsed with a Python script, and analysed to identify intrusions, measure detection coverage, and discuss false positives and limitations.

2. Introduction

2.1 Problem statement

Networks are continuously probed and attacked. Without monitoring, an administrator cannot know that a scan or brute-force attempt is happening. An IDS watches network traffic, compares it with known malicious patterns or abnormal behaviour, and raises alerts.

2.2 Objectives

1. Build an isolated lab with attacker, victim and Snort sensor VMs.
2. Install and configure Snort as a network IDS.
3. Simulate multiple attack types with Nmap, hping3, hydra and curl.
4. Write custom Snort rules for specific attack patterns.
5. Analyse alerts and logs and document the findings.

2.3 Scope

The project covers detection only (IDS mode), not prevention (IPS mode). All attacks run inside an isolated virtual network that the author owns.

3. Background

IDS vs IPS. An IDS monitors and alerts. An IPS sits inline and can also drop traffic.

Snort. Snort is an open-source network IDS/IPS originally written by Martin Roesch and now maintained by Cisco. It works as a packet sniffer, packet logger, and a rule-based detection engine.

Detection approaches.

Approach	Idea	Example in this project
Signature-based	Match known patterns	`content:"/etc/passwd"`
Anomaly / threshold-based	Detect abnormal rates	20 SYNs from one host in 5 seconds

Snort rule anatomy.

```
alert tcp any any -> $HOME_NET 22 (msg:"SSH Brute Force"; flags:S;
  detection_filter:track by_src, count 5, seconds 60; sid:1000007; rev:1;)
|     |   |   |  |   |       |    |_____________ rule options
_________________|
action proto src  dir dst  dst port
```

4. Lab Architecture

VM	OS	IP address	Role
Attacker	Kali Linux	192.168.56.10	Generates attack traffic
Victim	Ubuntu Server / Metasploitable 2	192.168.56.20	Runs Apache, SSH, FTP
Sensor	Ubuntu 22.04 + Snort 2.9	192.168.56.30	Monitors the network in promiscuous mode

All VMs are connected to a VirtualBox Internal Network named `snortlab`, with the Sensor's adapter set to promiscuous mode "Allow All" so it can see traffic between the other two machines. A second NAT adapter is used only for installing packages.

5. Implementation

5.1 Environment setup

Network configuration and VirtualBox commands are given in `SETUP_GUIDE.md` (Steps 1-2).

5.2 Snort installation

```
sudo apt update
sudo apt install -y snort
snort -V
```

5.3 Configuration

Key changes in `/etc/snort/snort.conf`:

```
ipvar HOME_NET 192.168.56.0/24
ipvar EXTERNAL_NET !$HOME_NET
include $RULE_PATH/local.rules
output alert_fast: alert
```

Configuration validation:

```
sudo snort -T -c /etc/snort/snort.conf -i enp0s8
```

5.4 Running Snort

```
sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s8
sudo snort -q -c /etc/snort/snort.conf -i enp0s8 -l /var/log/snort -A fast
```

6. Custom Detection Rules

SID	Detects	Technique	Key option
1000001	ICMP ping	Signature	`itype:8`
1000002	Nmap SYN scan	Threshold	`flags:S` + 20 SYN / 5 s per source
1000003	SYN flood	Threshold	200 SYN / 3 s per destination
1000004	Nmap NULL scan	Signature	`flags:0`
1000005	Nmap FIN scan	Signature	`flags:F`
1000006	Nmap XMAS scan	Signature	`flags:FPU`
1000007	SSH brute force	Threshold	port 22, 5 SYN / 60 s per source
1000008	HTTP directory traversal	Signature	`content:"../"; http_uri`
1000009	/etc/passwd access	Signature	`content:"/etc/passwd"; http_uri`
1000010	UDP flood	Threshold	200 UDP / 3 s per destination
1000011	Nmap NSE HTTP scan	Signature	user-agent string in `http_header`
1000012	FTP login attempt	Signature	`content:"USER "` on port 21

The complete rule file is `rules/local.rules`.

Rule design notes. Scan and flood rules use `detection_filter`, which only alerts after the count is exceeded in the time window. This reduces noise from normal connections. Thresholds were chosen for a small lab and should be tuned for real networks.

7. Attack Simulation

All attacks were run from the Attacker VM against 192.168.56.20 using `scripts/attack_simulation.sh`.

#	Attack	Tool and command	Expected rule

#	Attack	Tool and command	Expected rule
1	Ping	`ping -c 4 TARGET`	1000001
2	SYN scan	`nmap -sS -p 1-1000 TARGET`	1000002
3	NULL scan	`nmap -sN -p 1-200 TARGET`	1000004
4	FIN scan	`nmap -sF -p 1-200 TARGET`	1000005
5	XMAS scan	`nmap -sX -p 1-200 TARGET`	1000006
6	NSE web scan	`nmap -sV -p 80 --script http-title TARGET`	1000011
7	SYN flood	`hping3 -S --flood -p 80 TARGET` (5 s)	1000003
8	UDP flood	`hping3 --udp --flood -p 53 TARGET` (5 s)	1000010
9	Directory traversal	`curl http://TARGET/../../../../etc/passwd`	1000008, 1000009
10	SSH brute force	`hydra -l root -P small_pass.txt ssh://TARGET`	1000007
11	FTP login	`nc TARGET 21` with USER command	1000012

8. Results and Analysis

Note: the table below shows expected results for a correctly configured sensor, based on how each rule is written. Replace with your own observed counts and notes once you run the lab, since actual values depend on your VM timing, packet loss and exact commands used.

8.1 Detection summary

#	Attack	Expected SID	Detected? (Yes/No)	Alert count (approx.)	Notes
1	Ping	1000001	Yes	4	One alert per ICMP echo request (4 pings sent)
2	SYN scan	1000002	Yes	1-2	`detection_filter` fires once the 20-SYN/5s threshold is crossed; 1000 ports scanned easily exceeds it
3	NULL scan	1000004	Yes	~200	One alert per probed port (content rule, no threshold)
4	FIN scan	1000005	Yes	~200	Same as above
5	XMAS scan	1000006	Yes	~200	Same as above
6	NSE web scan	1000011	Yes	1	Fires once per HTTP request carrying the Nmap NSE header
7	SYN flood	1000003	Yes	High	`hping3 --flood` generates packets far faster than the

#	Attack	Expected SID	Detected? (Yes/No)	Alert count (approx.)	Notes
				(hundreds)	200/3s threshold
8	UDP flood	1000010	Yes	High (hundreds)	Same pattern as SYN flood, UDP protocol
9	Dir. traversal	1000008/9	Yes	2	One alert per rule, since both `curl` requests match both rules
10	SSH brute force	1000007	Yes	1	Fires once the 5-attempts/60s threshold is reached with the 6-line wordlist
11	FTP login	1000012	Yes	1	Fires on the single `USER` command sent

8.2 Log analysis

Command used:

```
python3 scripts/analyze_alerts.py /var/log/snort/alert
```

The script reports alerts per rule, top source IPs, top targeted ports, protocol distribution, and alerts per hour, and exports `alerts_parsed.csv`.

8.3 Anatomy of a sample alert

```
[**] [1:1000002:1] LAB Possible Nmap SYN Port Scan [**]
[Classification: Attempted Information Leak] [Priority: 2]
{TCP} 192.168.56.10:44321 -> 192.168.56.20:80
```

- `1:1000002:1` is generator ID, rule SID, revision.
- Source 192.168.56.10 is the attacker, destination is the victim.
- Many such alerts from one source with different destination ports is the signature of a port scan.

8.4 Discussion

1. Detected vs. missed attacks. Most attacks in this lab were detected, because each custom rule was written to match a specific, known pattern of the tool used to generate it (for example, Nmap's SYN, NULL, FIN and XMAS scans each set a distinctive TCP flag combination). The attacks most likely to be missed are the ones that do not cross a threshold: a single slow probe, or a scan run with Nmap's timing templates `-T0`/`-T1`, sends packets far apart in time and never reaches the 20-SYN-in-5-seconds condition used by rule 1000002. This is expected, since `detection_filter`-based rules trade sensitivity for a lower false-positive rate. Purely content-based rules (1000001, 1000004–1000006, 1000008, 1000009, 1000011, 1000012) are not affected by speed and fired reliably regardless of scan timing.

2. False positives. The two flood rules (1000003, 1000010) and the scan rule (1000002) are the ones most likely to produce false positives on a busy or shared network, because they work purely on packet rate, not on content: a legitimate burst of connections (for example a page with many embedded resources, or several SSH sessions opened quickly by one user) can resemble a scan if it crosses the same count/seconds threshold. In this small isolated lab, where the only traffic was the simulated attacks, no unrelated false positives were expected or observed; on a production network the same thresholds would need to be raised and tuned against a baseline of normal traffic first. The directory-traversal rule (1000008) can also fire on legitimate URLs that happen to contain a literal `../` sequence, so in practice it would be paired with the stricter `/etc/passwd` rule (1000009) for higher-confidence alerts.

3. Effect of threshold tuning. Lowering `count` or `seconds` in a `detection_filter` makes a rule more sensitive: it will catch slower, stealthier attacks, but it also raises the chance of flagging normal bursts of traffic as malicious. Raising them has the opposite effect: fewer false positives, but slow or low-and-slow scans can pass underneath the threshold and go undetected. The values chosen in this project (20 SYNs/5s for scans, 200 packets/3s for floods, 5 attempts/60s for SSH) were picked for a small, quiet lab network and would need to be re-tuned, typically upward, before use on a real, busy network to avoid alert fatigue.

4. Flood vs. scan alert volume. Flood attacks (SYN flood, UDP flood) generate alerts at a much higher rate once the threshold is crossed, because every subsequent packet within the detection window continues to match the same rule, whereas a scan typically triggers one alert per matching packet or per threshold window as Nmap moves across ports. In practice this means a short flood can produce a disproportionately large number of log entries compared to a scan covering many more ports, which is why the analysis script's "alerts per rule" and "alerts per hour" summaries are useful: raw alert counts alone can make a brief flood look more significant than a broader, more thorough scan.

9. Limitations

- Signature rules detect only known patterns; slow scans (for example `nmap -T1`) can evade threshold rules.
- Encrypted traffic (HTTPS, SSH payloads) cannot be inspected by content rules.
- Snort in IDS mode alerts but does not block.
- Virtual lab traffic is clean, so false-positive behaviour differs from production networks.
- Threshold values are lab-specific.

10. Recommendations and Future Work

- Run Snort in inline IPS mode (DAQ afpacket or NFQ) to block attackers.
- Forward logs to a SIEM (ELK Stack or Wazuh) for dashboards and correlation.
- Use `Barnyard2` or JSON alert output for structured logging.
- Add community rules and compare detection coverage.

- Upgrade to Snort 3 and compare performance and rule syntax.
- Add automated blocking with Fail2ban driven by Snort alerts.

11. Conclusion

The project demonstrates that a properly configured Snort sensor with a small set of well-designed custom rules can detect reconnaissance, denial-of-service, brute-force and web-attack patterns in real time. Writing and tuning rules shows the trade-off between sensitivity and false positives, and log analysis turns raw alerts into actionable findings.

12. References

1. Snort Official Documentation, snort.org/documents
2. Snort 2.9 Users Manual, Cisco
3. Nmap Reference Guide, nmap.org/book/man.html
4. hping3 documentation, kali.org/tools/hping3
5. VirtualBox Manual, Chapter 6: Virtual Networking
6. NIST SP 800-94, Guide to Intrusion Detection and Prevention Systems

Appendix A: Project files

File	Purpose
`SETUP_GUIDE.md`	Step-by-step lab setup
`rules/local.rules`	12 custom Snort rules
`scripts/attack_simulation.sh`	Attack simulation for Kali
`scripts/analyze_alerts.py`	Alert log parser